VINFAST

Specification

Part_Number

Specification Book

Supersedes Edition (Doc. No.) (mm.yyyy)

1. Draft: dd.mm.yyyy

Scope

Applicable Document of Part 1

Common Electrical Specification for Supplier
CES Coding Specification

Range Of Validity

VinFast

Author: Lutz Neubacher

The reproduction, distribution and utilization of this document as well as the communication of its contents to others without express authorization is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. Nothing in this standard supersedes applicable laws and regulations.
G10100-101f

Approval Confirmed By

	Dept.Code	Name	Signature	Date

Change Order KAU-Number

Index

Scope **1**
Range Of Validity **1**
Approval Confirmed By **2**
Index **3**
1 General Information **4**
1.1 Language 4
1.2 Change History 4
1.3 Versioning 4
2 Introduction **4**
2.1 Scope 4
2.2 Terms and Definitions 5
2.3 Abbreviations 5
3 Specification **6**
3.1 Classification of ECUs 6
3.2 Coding data content 7
3.3 Coding data format 9
3.4 The structure of the VOD 9
3.5 The structure of the CDS 10
3.6 ECU coding states 11
3.7 Coding sequences 14
3.8 Update of vehicle order data 15
3.9 Mismatch detection strategy 19
4 Standards and Documents **22**
4.1 Standards 22
4.2 Referenced Documents 22
5 Lists **23**
5.1 List of Tables 23
5.2 List of Figures 23
6 Further Applicable Documents **24**

1 General Information

1.1 Language

Req ID	11960	Checkin date	21.03.2018

This Specification is made in English version.

1.2 Change History

Req ID	11961	Checkin date	27.03.2018

Version	Chapter	Brief Description	Date	Responsible person
0.0.1	all	first draft	*21.03.2018*	L.Neubacher
1.0.0	all	reworked	*24.03.2018*	C. Lechner
1.0.1	Coding Data Content	update Coding Variants	*27.03.2018*	L.Neubacher

Table 1: Change History

1.3 Versioning

Req ID	11962	Checkin date	08.03.2018

To determine the actual status of the present specification, see the following table:

Version	State	MSE Gate/Milestone
0.1.0 - 0.49.9	DRA „Draft“	FUP
0.50.0 - 0.99.9	ICS „Inquiry Concept Specification“	ILF
1.0.0 - 1.99.9	ISS „Inquiry Serial Specification“	AF
3.0.0	FIN „Final“	TA

Table 2: Rules for Versioning

Version Digit description:

First Digit: Major Release (Milestone Release)
Second Digit: Minor Release (Function description change)
Third Digit: No functional content change (e.g. spelling correction)

2 Introduction

2.1 Scope

Req ID	11964	Checkin date	23.03.2018

This document describes the basic requirements for coding of ECUs and is intended for ECU developers.

In the past, different part number is defined to distinguish the different configuration, but this method would lead to increase cost of validation, logistic, and errors. The goal of coding function is to reduce the ECU variants which have the same hardware.

Coding of ECUs is an important task for adapting ECU behaviour in a vehicle. While applying its general functionality via flash programming the application software to it, the ECU's final behaviour may be fine-tuned by configuration data provided by coding. Therefore, it is common practice to do a re-coding after flashing new program software to an ECU.

Compared to a flash programming sequence the amount of data transferred from tester to an ECU in a coding sequence is usually much smaller, but the common goal of coding and flash programming is to store data in ECU's non-volatile memory.

The behaviour of an ECU without coding function is solely defined by its application software and needs not be adapted further in anyway. These ECUs are not affected by this specification.

2.2 Terms and Definitions

Req ID	11967	Checkin date	

Not applicable

2.3 Abbreviations

Req ID	11968	Checkin date	22.03.2018

UDS	Unified Diagnostic Services
VOD	Vehicle Order Data
SAE	Society of Automotive Engineers (www.sae.org)
CRC	Cyclic Redundancy Check
ODX	Open Diagnostic Data Exchange (ASAM MCD-2 D)
OBD	On Board Diagnostic
MCD	Measurement Calibration and Diagnostics
CDS	Coding Data String
ISO	International Organization for Standardization (www.iso.org)
ECU	Electronic Control Unit
ASAM	Association for Standardisation of Automation and Measuring Systems (www.asam.net)
CAN	Controller Area Network
DTC	Diagnostic Trouble Code
VIN	Vehicle Identification Number
AS	After sales

Table 3: Abbreviations

3 Specification

3.1 Classification of ECUs

Req ID	11970	Checkin date	22.03.2018

Concerning coding capabilities, ECUs in a vehicle are classified into following categories:

0. ECUs without coding capabilities
1. ECUs with coding capabilities only for ECU adaption
2. ECUs with coding capabilities for vehicle adaption and ECU adaption

The behaviour of an ECU of category 0 is solely defined by its application software and needs not be adapted further in any way. These ECUs are not affected by this specification.

An ECU of category 1 is usually more complex than an ECU of category 0. Its behaviour is mainly defined by its application software, but can be adapted by ECU coding data, which is called Coding Data String (CDS) in the following (e.g. a Body Control Module ECU can be adapted for daytime running lights, which are mandatory in some countries).

An ECU of category 2 should store vehicle coding information, that is relevant for multiple ECUs of a vehicle. If a ECU is defined to store the VOD (called Vehicle Order Data) it must support mechanisms to write, hold and read vehicle coding information.
The definition, which ECU has to store the VOD, must be done by the OEM.

Remark: For this Project the Gateway and IPC should store the VOD.

The complete Vehicle Identification Number (VIN with17 digits) is also stored in this ECUs, but is not considered as part of vehicle coding data, since it is immutable.

In close coordination with ECU manufacturers, the OEM must determine the coding category of each ECU inside a vehicle. An ECU can either belong to

- category 0 or
- category 1 or
- category 2

ECUs of category 2 support the mechanism of category 1 ECUs and mechanism which are needed to indentify a vehicle.

For redundancy reasons it is also highly recommended to have two ECUs belonging to category 2 in a vehicle.

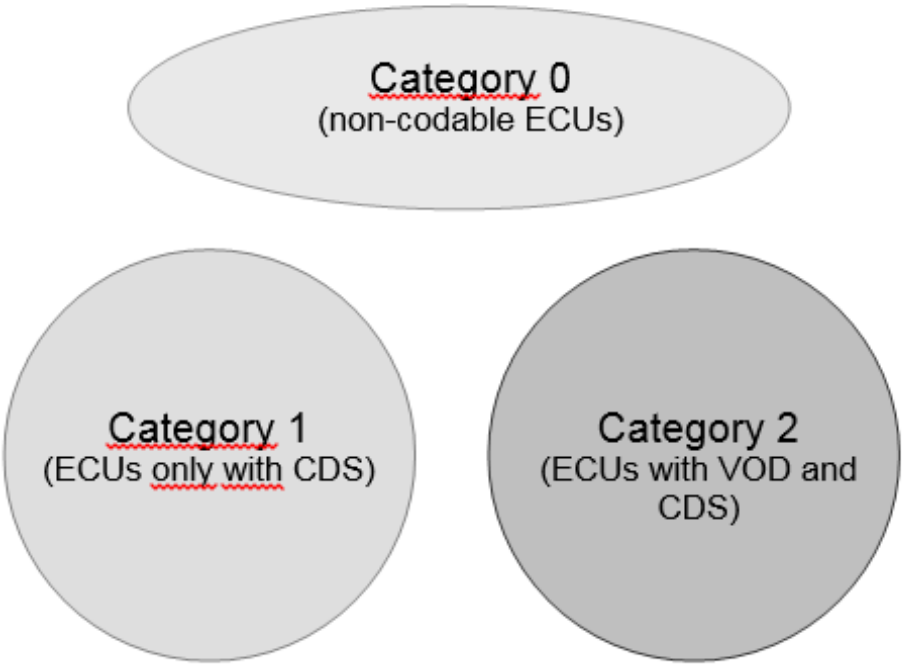


Figure 1: Classification of ECUs

3.2 Coding data content

Req ID	11971	Checkin date	27.03.2018

The content of Vehicle Order Data (VOD) shall include information about topology and configuration of the whole vehicle. It would include the whole vehicle variant information like vehicle type variants, vehicle variants, (yearly) model revision, engine type, country specific behaviour of ECU, vehicle equipment dependant variant (two doors or four doors), etc.

Some example about VOD definition like followings:

Byte position	Bit position	Function description	Value table
Byte 0	Bit7-bit0	vehicle type	0x00=Not recognised 0x01=Sedan 0x02=SUV 0x03=reserved 0x04=reserved 0x05=reserved :
Byte 1	Bit15-bit8	vehicle variants	0x00=Not recognised 0x01=Base Content 0x02=Style Package 0x03=Tech Package 0x04=Comfort Package 0x05=Russia AddOn Content

			:
Byte 2	Bit23-bit16	model revision	0x00=Not recognised 0x01=2018 0x02=2019 0x03=2020 0x04=2021 :
Byte 3	Bit31-bit24	engine type	0x00=Not recognised 0x01=N20 Low Power 0x02=N20 High Power 0x03=reserved 0x04=reserved 0x05=reserved 0x06=reserved

Table 4: Example for the contents of VOD

VOD include factory VOD and current VOD. Factory VOD only can be written once during the manufacturing process at the BRI side. Once written, this can't be changed any more. Current VOD shall be same as factory VOD if function of the whole vehicle is never been changed. Update VOD is allowed only after approved by the AS server.

The content of Coding Data String (CDS) is only ECU specific, since requirements of the individual ECU memory and the corresponding application software have to be taken into account. It is basically implemented a table in an ECU, in this table each value modifies one specific ECU behaviour. The values are transmitted to the ECU as a string ("CDS" coding data string) and stored in the ECU in Non volatile RAM, protected by Checksum.
Define the VOD and map a vehicle's topology and configuration accordingly. VOD definition could references to document [1] VOD definition.

Some examples about CDS definition in BCM like followings:

Byte position	Bit position	Function description	Value table
Byte 0	Bit1-bit0	Welcome light	0x00=Not recognised 0x01=with welcome light 0x02=without welcome light
Byte 0	Bit3-bit2	Daytime running light	0x00=Not recognised 0x01=with daytime running light 0x02=without daytime running light
Byte 0	Bit5-bit4	Transmission type	0x00=Not recognised 0x01=MT 0x02=AT
Byte 0	Bit7-bit6	Rear wiper	0x00=Not recognised 0x01=with rear wiper 0x02=without rear wiper
Byte1	Bit9-bit8	Anti pinch window	0x00=Not recognised 0x01=with Anti pinch window 0x02=without Anti pinch window
Byte 1	Bit11-bit10	ESS	0x00=Not recognised 0x01=with ESS 0x02=without ESS
:	:	:	:

Table 5: Example for the contents of CDS

The last seven characters of the VIN (called Short-VIN in the following) are part of each Coding Data String.
This is necessary e.g. to check the exchange of ECUs between different vehicles.
The reaction in the coding process, if an ECU is not valid for the vehicle, has to be defined by Brilliance.

The meaning of the ECU individual coding options is described based on the function list in document [2] CDS definition

The CDS of the single ECU are derived from VOD that describes the build state of the complete vehicle. An external off board diagnostic IT system would calculate the CDS for the individual ECU from the VOD automatically, it is not the part of this specification. (Reference to document [3] off board coding system specification)

3.3 Coding data format

Req ID	11972	Checkin date	23.03.2018

The coding data format, i.e. format of Coding Data String and Vehicle Order Data, to be used in diagnostic communication (please refer to ISO 14229-1) shall be implemented in vehicle's ODX data.

For additional details please refer to the document [4]

For verification purposes, it is also required to transmit an 8-bit-CRC value, calculated from the content of the coding data to be stored in the ECU's non-volatile memory (not including the data length byte and short VIN number, only relevant with data of CDS and VOD).

The CRC algorithm to be used for this calculation is defined by SAE (please refer to SAE-J1850 for details), using the division polynomial $x^8 + x^4 + x^3 + x^2 + 1$.

The ECU manufacturer can freely define the storage layout of CDS in an ECU's non-volatile memory. It shall be guaranteed, that the transmitted CDS is stored unaltered in memory and detection of coding data corruption due to memory faults is possible in a reliable way. It is highly recommended, to use the CRC algorithm defined in SAE-J1850 for this purpose, too, to simplify the check of unaltered coding data storage in memory.

3.4 The structure of the VOD

Req ID	11973	Checkin date	23.03.2018

DID	VOD length Byte	Vehicle Order Data (topology and configuration)	CRC
0xF111 0xF112	1 Byte	n Byte (max. 255)	1 Byte

Table 6: Structure of the VOD

The two VOD is stored in one or two relevant ECU of the vehicle. The ECU manufacturer has to make sure sufficient space memory in the ECU to store the VOD.

The Vehicle Order Data length Byte describes the numbers of used Bytes in the Vehicle Order Data string including CRC bytes. With 1 Byte length Byte it is theoretically possible to use 255 Byte in the Vehicle Order Data string.

Remark: The length of the VOD has an impact of the needed VOD writing time.

Recommendation: shorter VOD is faster to write.

The n Byte Vehicle Order Data (topology and configuration information) describe the used options of the whole vehicle.

The options can be differed from variant to variant. The VOD of the vehicle describe only the options of the selected vehicle. This means, the VOD is vehicle specific and shall be generated especially for any vehicle.

Additional information about the data content of the VOD is described in the document [4]

3.5 The structure of the CDS

Req ID	11974	Checkin date	22.03.2018

DID	Data length Byte	Short VIN	ECU Coding Data String	CRC
0xF108	2byte (only used 10bit)	7 Byte	n Byte (max.1023)	1 Byte

Table 7: Structure of the CDS

The CDS is the description of the coding option of the ECU. The ECU manufacture has to make sure sufficient space memory in the ECU to store the CDS. The CDS is ECU specific and include all necessary information to code or parameterizes the ECU in the vehicle.

If the length of CDS for some ECU is longer than 1023 byte, this data must be written into ECU by flashing. And written by service 0x2E is not allowed, because it will cost too much time, and also a CRC-8 does not protect such a big field very well.

The length of the CDS is depends from the amount and content which will be coded or parameterized, including CRC.

Remark: The length of the CDS has an impact of the needed coding time.

Recommendation: shorter CDS is faster to write.

The field Short-Vin (7 Bytes) includes the last 7 values of the complete Vehicle Identification Number (VIN).

The field ECU configuration information describes the coding options of the affected ECU. CDS switches a special function (the option) in the ECU to ON or OFF.

The internally data content of the CDS is defined and controlled by component engineer and ECU supplier. The data content of CDS could be also used, to describe and write ECU specific parameters. The content of CDS is described in the document [2].
How to restore some data such like learning data or data input by customer, by coding or other method should be defined by ECU supplier.
Remark: The ECU specific CDS is also depends on the configuration of the whole vehicle. This means that the VOD (for the vehicle) and CDS (for the different ECUs of the vehicle) are very closely linked.

3.6 ECU coding states

Req ID	11975	Checkin date	22.03.2018

There are three cases for a codeable ECU which needs coding (CDS) process:

1. ECU with DTC (not coded): ECU is a new one and going to be mounted on the vehicle first time, including ECU replacement in AS.
2. ECU with DTC (coding invalid): ECU is coded, but the data of CDS is wrong or haven't been checked yet. So recoding process is needed.
3. ECU coded: ECU have been coded rightly at all, but some option function is changed, so recoding process is needed too.

Figure "Coding states of an ECU" illustrate a transition diagram of ECU states related to coding, that shall be implemented by each ECU belonging to category 1 or category 2. There are four states when ECU during in the coding process as followings:

1. ECU is not coded (CodingStateFlag=NotCoded).
2. ECU is not checked (CodingStateFlag=CodingNotChecked).
3. Coding is invalid (CodingStateFlag=CodingInvalid).
4. Coding is valid (CodingStateFlag=CodingValid).

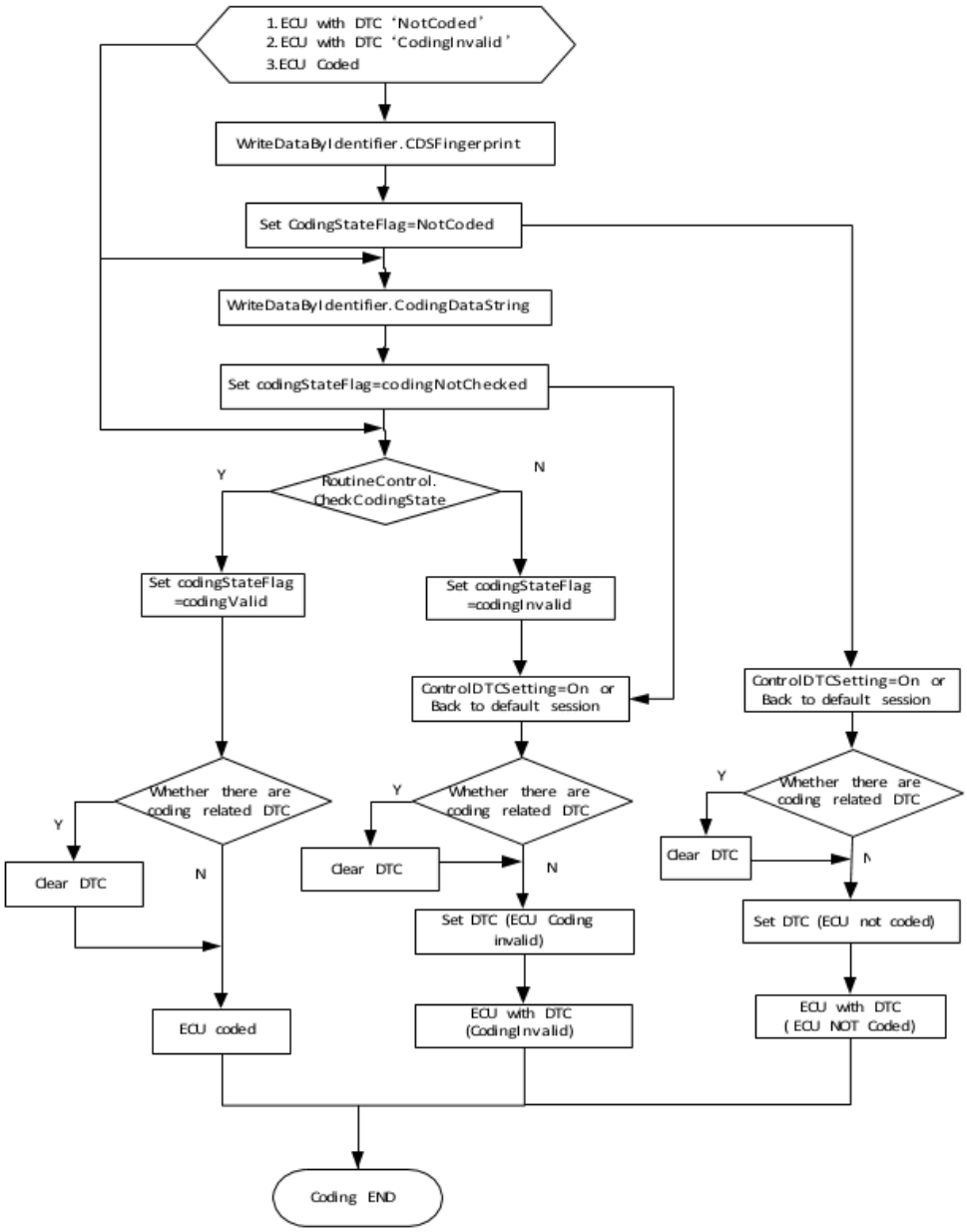


Figure 2: Coding states of an ECU

After ECU enters coding process and is unlocked, ECU is necessary to initiate the coding properly, what is achieved by sending the diagnostic service **WriteDataByIdentifier.CDSFingerprint**. The information provided to the ECU by this diagnostic service is also used to keep track of who performed the last coding. The CDSFingerprint can refer to the document [4] general DID definition.

After this service, there is an ECU internal step to set CodingStateFlag=CodingNotCoded. And now the ECU is considered being "Not coded". To store this coding data the diagnostic services **WriteDataByIdentifier.CodingDataString** has to be sent to the ECU. The data transmitted in these diagnostic services is written directly to the ECU's non-volatile memory.

The ECU is coded now, but whether the coding is valid or not is not yet determined, therefore the ECU is considered being "Not checked" (CodingStateFlag=codingNotChecked). It is possible (although not very reasonable) to do a re-coding of the ECU while being in this state. To finish the coding of an ECU, its coding state has to be requested by sending the diagnostic service **RoutineControl.CheckCodingState**.(This routine ID can refer to document [5].)

After receiving this diagnostic service, the ECU has to

- Perform a memory check to detect corrupt coding data. E.g. re-calculate the CRC of the stored coding data and compare to the CRC which was saved during the writing of the coding data
- Return a list of identifier pointing to invalid coding data.(CodingStateFlag= CodingInvalid)

Only returning an empty list (response = 71, 01, 02, 00) means that the ECU's state is "Coding valid". (CodingStateFlag= codingValid)
Following identifiers have to be supported:

- 0xFxxx: codingFingerprint Invalid, if content of DID codingFingerprint is all 0x00 (HEX);
- 0xFxxx: CodingDataString Invalid, if Short-VIN compared with message which is sent by ECU on network is not identical (only ECU category 2), or CRC of Coding Data String is not correct,or the actual data length isn't match the definition byte which after the DID number;

In case of an ECU of category 1, the ECU has to perform a comparison of the stored Short-VIN to the last 7 bytes of the complete VIN from the network.
BCM send short VIN number on CAN network, ECUs who have coding function shall receive it and check it internally.
The steps

1. **WriteDataByIdentifier.CDSFingerprint**
2. **WriteDataByIdentifier.CDS**
3. **RoutineControl.CheckCodingState**

are part of each coding sequence given in section 3.7 describing further requirements for admissible ECU coding.
After ECU back to default session or reactive the DTC storage function, DTC (coding invalid) would be stored if the CodingStateFlag is codingInValid or CodingNotChecked. DTC (ECU not coded) would be stored if the CodingStateFlag is NotCoded.
In case of any failure or not correct coding state, the default value which is defined in "M8x VOD and CDS definition "shall be used.
If some coding function is precondition for any assembly work, BRI will request to be defined in component diagnostic specification.
***Remark*:** The transition from state "Coding valid" to "Coding invalid" may occur after the coding sequence finished, e.g. due to a memory faults.

3.7 Coding sequences

Req ID	11976	Checkin date	23.03.2018

To write CDS into ECU, diagnostic service sequence which is structurally in the figure "coding sequence of ECU´s" shall be supported by their application software.

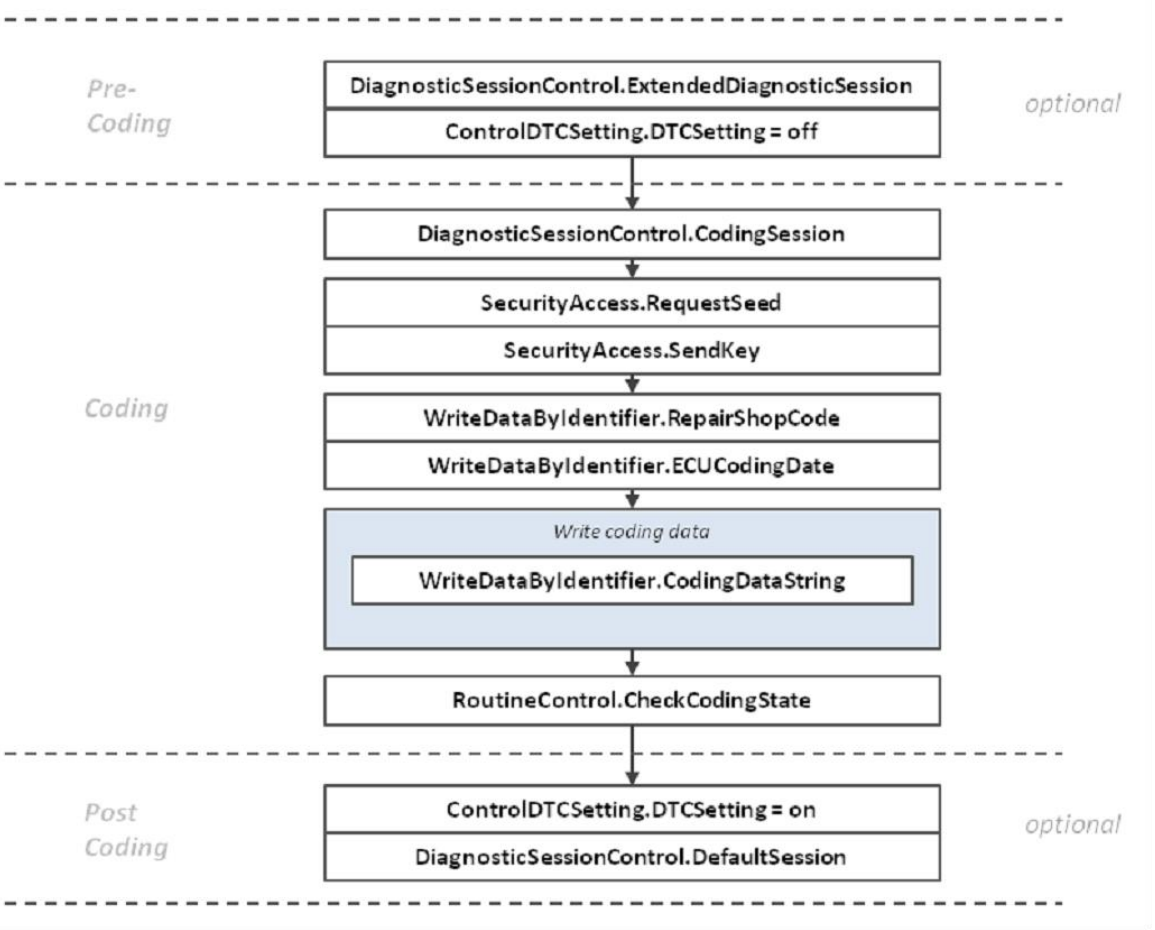


Figure 3: Coding sequence for ECUs (storing Coding Data String)

To initialize ECU coding, the ECU shall exit the default diagnostic session and switch to the extended diagnostic session. This is achieved by sending the diagnostic service **Diagnostic- SessionControl.ExtendedDiagnosticSession**.
After having entered the extended diagnostic session, it is possible to turn off the storage of DTC Data in ECU's error memory during the coding phase. The diagnostic service **ControlDTCSetting.DTCSetting=off** can be sent to the ECU for this purpose. Only when storing a new Coding String this step is optional and can be skipped, since only this ECU's application software is affected by the coding data.
Storing new coding data in an ECU's non-volatile memory may only be possible while being in the coding session. Transition to the coding session may only occur from the extended

diagnostic session by sending the diagnostic service **DiagnosticSessionControl.Coding-Session**

to the ECU.

An ECU shall be protected against unauthorized coding by a Seed-Key-Mechanism. The diagnostic services **SecurityAccess.RequestSeed** and **SecurityAccess.SendKey** have to be used the algorithm in the document [6].
As mentioned in section (ECU coding states) the storage of coding data in the non-volatile memory is only possible for an ECU while being in the correct coding state. Therefore, the diagnostic service **WriteDataByIdentifier.CDSFingerprint** shall be sent to the ECU to track when and who write the coding data.
By sending the diagnostic service **WriteDataByIdentifier.CodingDataString**, the coding data is stored to the ECU's non-volatile memory. To check whether the storage of the transmitted coding data was successful, the diagnostic service **RoutineControl.CheckCodingState** shall be sent to the ECU (see also section ECU coding states).
Finalizing the coding of an ECU properly is achieved and switch to extended session by sending the diagnostic service **DiagnosticSessionControl.extendedSession** to the ECU.
And then ECU will be re-activated the DTC storage (if it was deactivated before) by sending the diagnostic service **ControlDTCSetting.DTC-Setting=on** and switching back to the default diagnostic session by sending the diagnostic service **DiagnosticSessionControl.DefaultSession** to the ECU.
***Remark*:** It is not necessary to send the diagnostic service **ECUReset** to the ECU in order to consider the new coding data as active. The ECU will have to use the coding data as soon as it is stored in its non-volatile memory, but the ECU's shall executed a failsafe mode while being in coding state "Not checked". Every ECU shall give the proposal to define the failsafe mode.

3.8 Update of vehicle order data

Req ID	11977	Checkin date	23.03.2018

VOD shall be written after coding sequence. Only category 1 ECU replacement doesn't need to update the VOD. There are two use cases of the VOD written:

1. Both factory VOD and current VOD are needed to write into the category 2 ECU. In this case, maybe either ECU is mounted on the vehicle first time, factory VOD and currently VOD are the default value and need to write into ECU, Or Both factory and current VOD are invalid, rewritten factory VOD and current VOD is needed. The figure "Factory VOD and current VOD written" illustrates the state transition for this case.

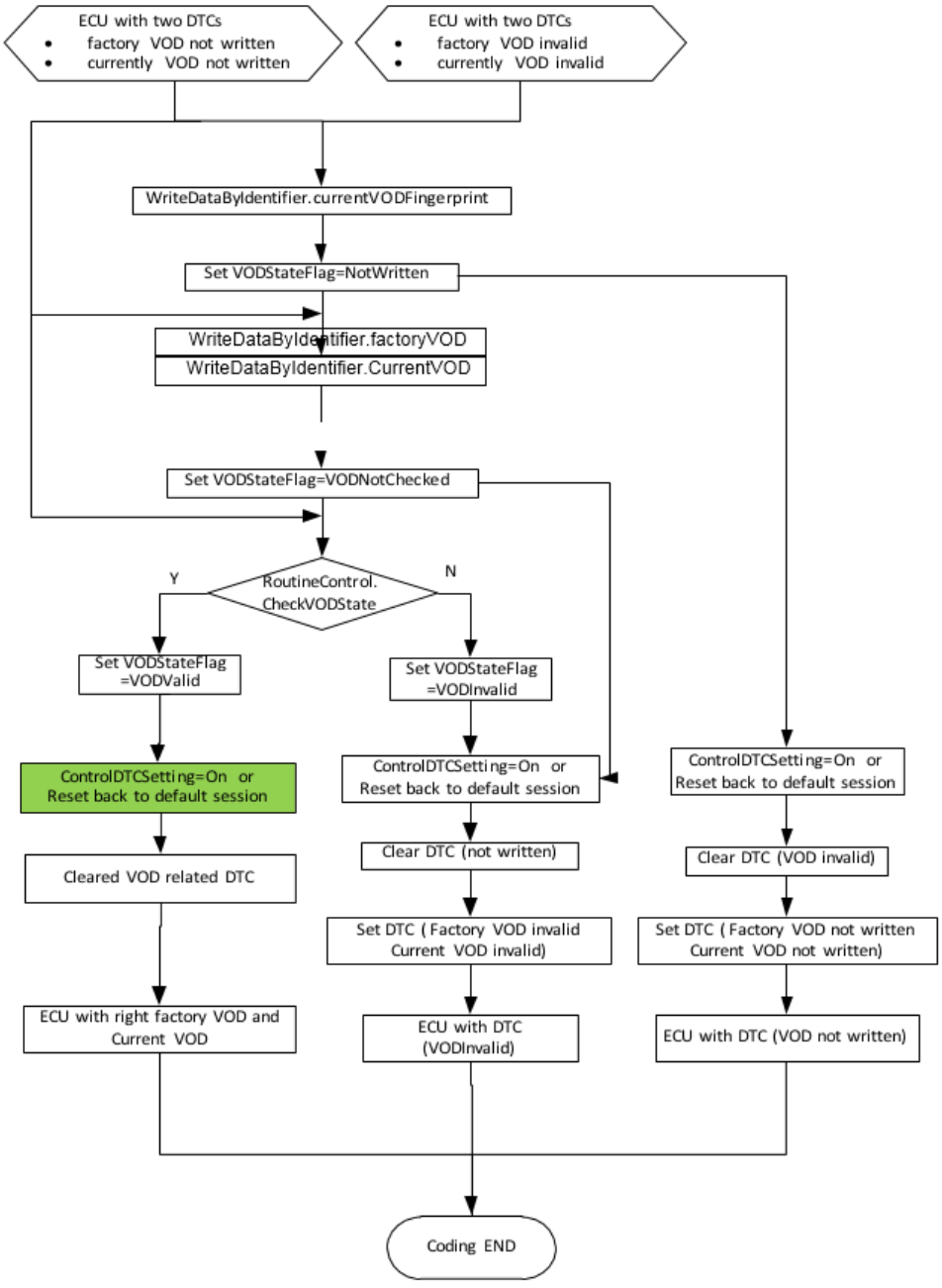


Figure 4: Factory VOD and current VOD written

2. Only the current VOD need to be updated. 1) ECU with valid current VOD, but intend to change vehicle option function, a new current VOD is needed. 2) ECU with invalid current VOD or current VOD not written, a new current VOD is needed. Figure "current VOD written" illustrates the state transition for this case.

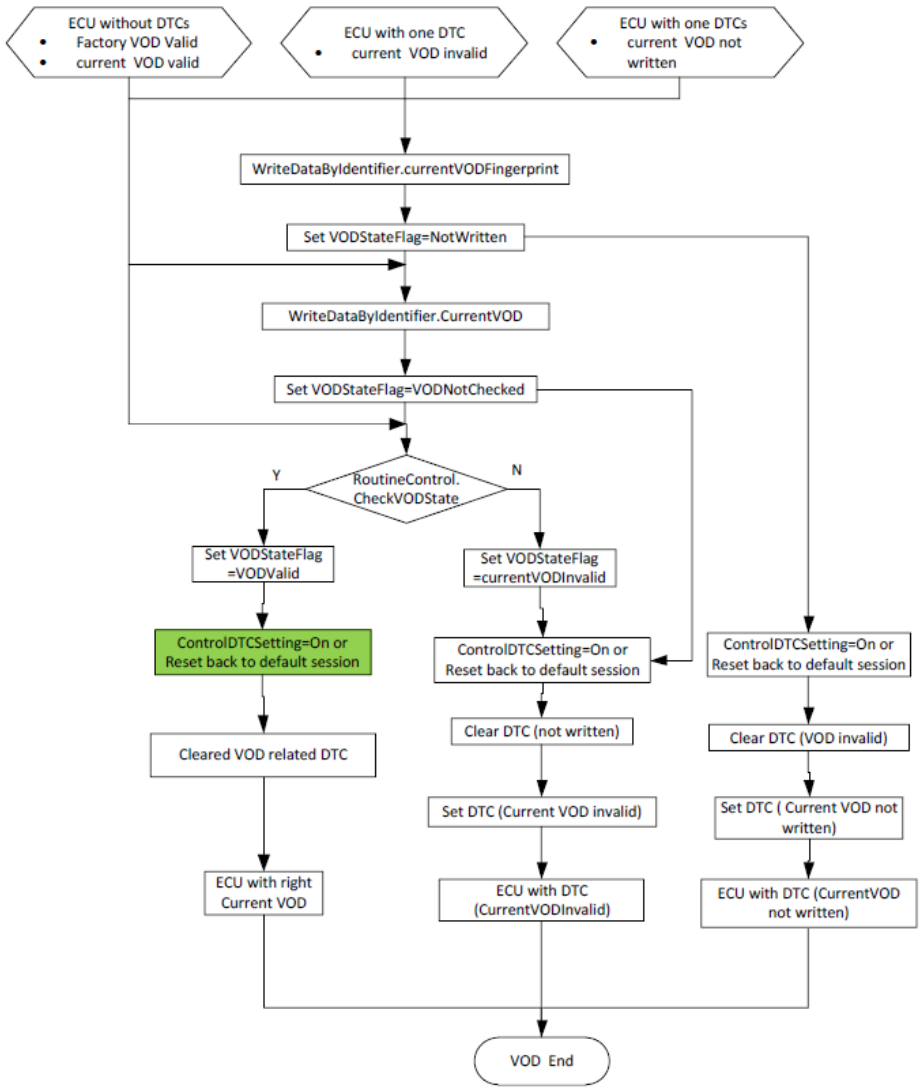


Figure 5: Current VOD written

The strategy of the state transition of VOD written is similar as the CDS written. The updating of a Vehicle Order Data affects multiple ECUs (all of category 2) simultaneously. Therefore, the coding sequence given in Figure Coding sequence for ECUs shall be slightly extended in that way, that each phase (namely Pre-Coding, Coding, Post-Coding, separated by dashed lines) shall be executed for each ECU of category 2 like shown in Figure "Coding sequence for updating a vehicle's Vehicle Order Data"
The reason is that the Vehicle Order Data shall be stored in multiple ECUs for redundancy reasons and has to be kept synchronized within a vehicle. ECUs of category 2 should therefore compare their stored Vehicle Order Data against each other and store a DTC on detecting a mismatch (detailed information can refer to chapter 3.9). Since not all ECUs of category 2 can be updated in parallel, there will be an unknown period of time where this mismatch may occur. Turning off the DTC storage during distributing a new Vehicle Order is therefore necessary. DID of currentVODFingerprint should be refer to document [4], and routine ID of checkCurrentVODState, checkfactoryVODState should refer to document[5],

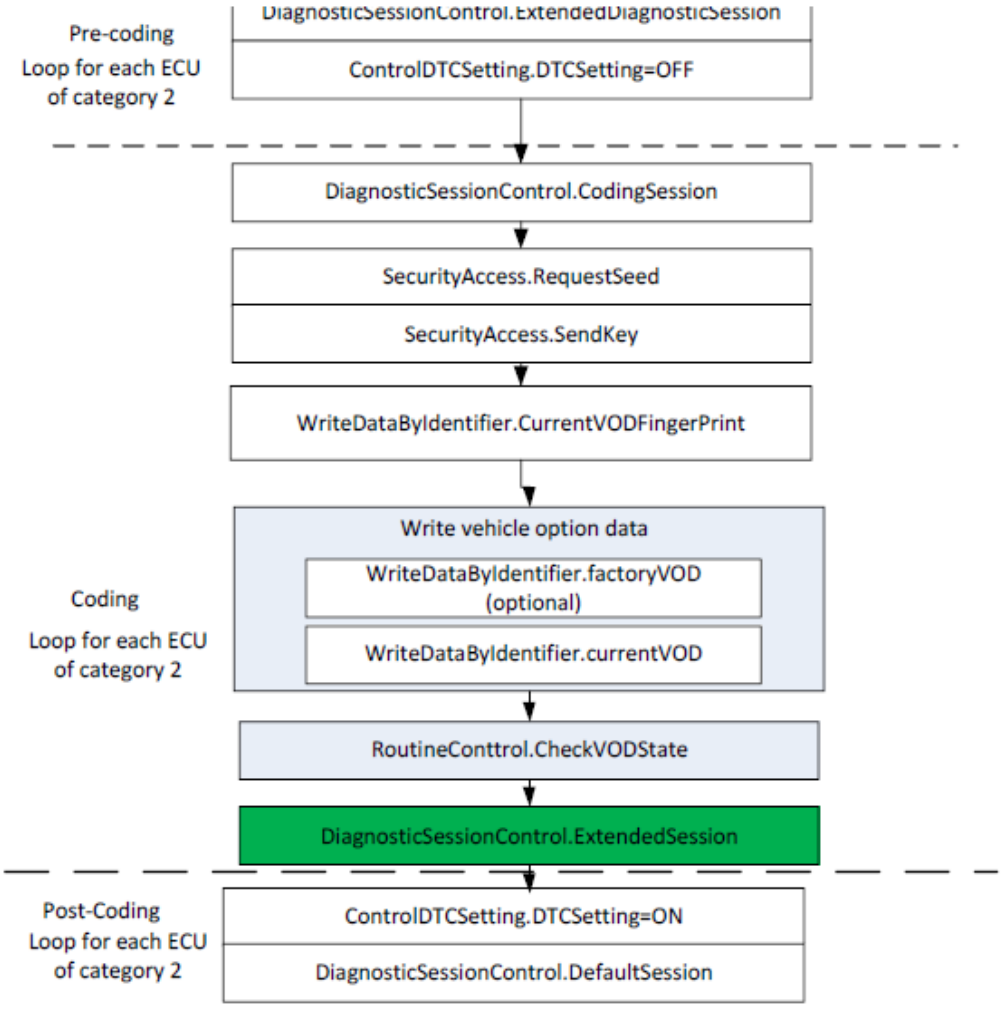


Figure 6: Coding sequence for updating a vehicle's Vehicle Order Data

About the implementation of service **RoutineControl.CheckVODState** can refer to document [5]. After receiving this diagnostic service, the ECU will do the same work as check coding state:

- Perform a memory check
- Return a list of identifier pointing to invalid coding data.(CodingStateFlag= CodingInvalid)

Only returning an empty list (response = 71, 01, 02, 01) means that the ECU's state is "VOD valid". (VODStateFlag= VODValid)

Following identifiers have to be supported:

- 0xFxxx: VODFingerprint Invalid, if content of DID VODFingerprint is all 0x00 (HEX);
- 0xFxxx: FatoryVOD Invalid, if CRC FactoryVOD is not correct;
- 0xFxxx: CurrentVOD Invalid, if CRC CurrentVOD is not correct;

3.9 Mismatch detection strategy

Req ID	18627	Checkin date	22.03.2018

One of Category 2 ECUs ECU_A can map the new VOD to their CAN messages, these VOD_message are eight byte block, the first byte is block number and VODUpdatedFlag , and the following seven byte is the VOD information, message layout should refer to the follow table:

Byte 7	Byte 6	Byte 5	Byte 4	Byte 3	Byte 2	Byte 1	Byte 0
VOD_byte6	VOD_byte5	VOD_byte4	VOD_byte3	VOD_byte2	VOD_byte1	VOD_byte0	Block 0 and flag
VOD_byte13	VOD_byte12	VOD_byte11	VOD_byte10	VOD_byte9	VOD_byte8	VOD_byte7	Block 1 and flag
VOD_byte20	VOD_byte19	VOD_byte18	VOD_byte17	VOD_byte16	VOD_byte15	VOD_byte14	Block 2 and flag
VOD_byte27	VOD_byte26	VOD_byte25	VOD_byte24	VOD_byte23	VOD_byte22	VOD_byte21	Block 3 and flag
VOD_byte34	VOD_byte33	VOD_byte32	VOD_byte31	VOD_byte30	VOD_byte29	VOD_byte28	Block 4 and flag

Table 8: VOD message

ECU_A shall transmit all 35 bytes VOD information by 5 frame within the scope of 1 second and repeat them every 1 second. The transmit period of VOD_message is 200ms.

	Time slot 0ms	Time slot 200ms	Time slot 400ms	Time slot 600ms	Time slot 800ms
0s	Block 0	Block 1	Block 2	Block 3	Block 4
1s	Block 0	Block 1	Block 2	Block 3	Block 4
……					

Table 9: VOD block schedule

ECU_A would check internally whether DID of VODs has been changed or not by the off board tester. If yes, ECU_A would set the flag VODUpdatedFlag=1 and then map the new VOD to their CAN messages.

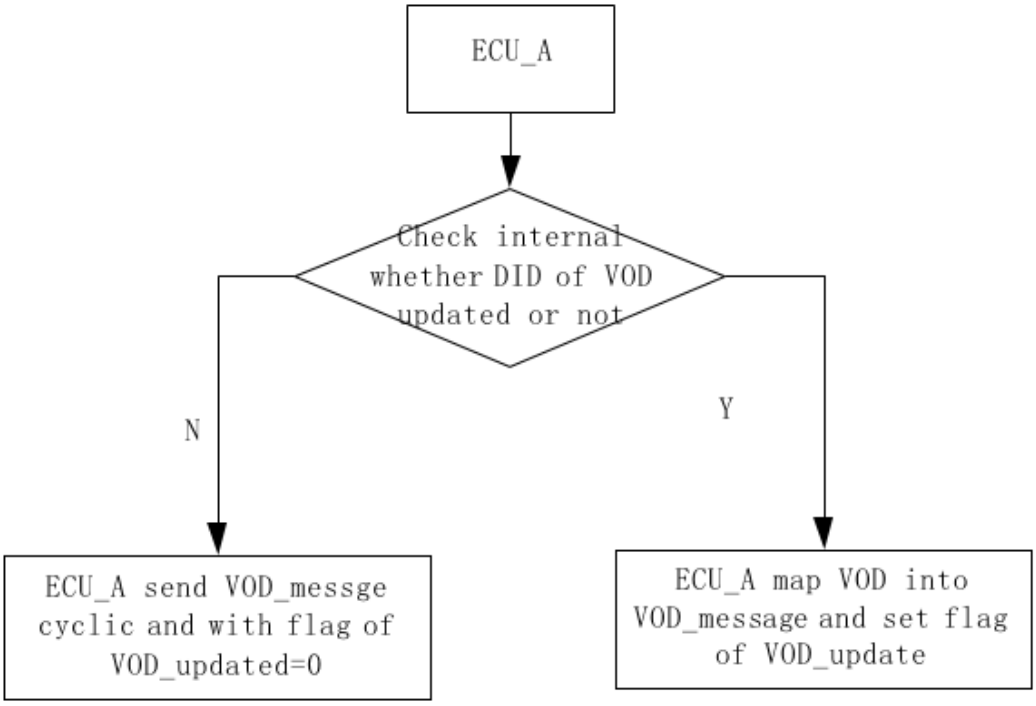


Figure 7: Strategy in ECU_A for mismatch detection

ECU_B will compare the VOD which received from network with VOD which stored in their own non-volatile memory during the following two cases:

1. VODUpdatedFlag=1
2. VOD which stored in its own memory has been changed.

If they are different, DTC that VOD doesn't match would be stored.

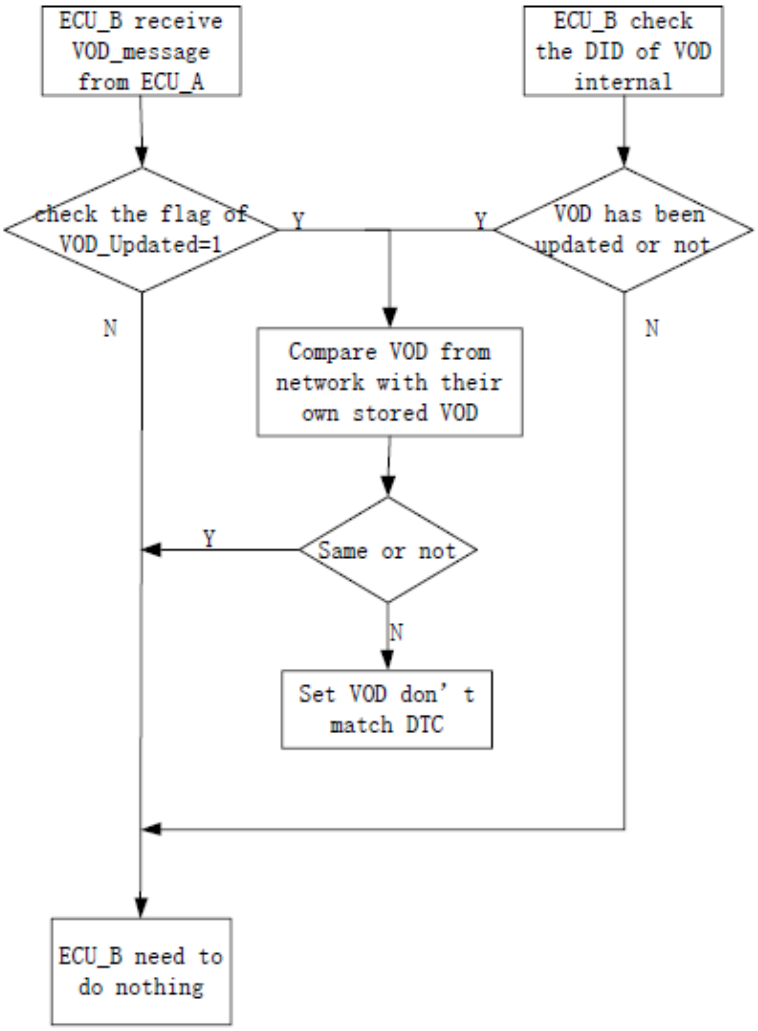


Figure 8: Strategy in ECU_B for mismatch detection

4 Standards and Documents

4.1 Standards

Req ID	18803	Checkin date	22.03.2018

Follow Standards and Documents are binding:

Document	Date	Version	Title
ISO 14229-1	Apr 2007	-	Road vehicles – Unified diagnostic services (UDS) *Part 1: Specification and requirements*
SAE-J1850	Mar 1998	-	Data Communications Network Interface

4.2 Referenced Documents

Req ID	11979	Checkin date	23.03.2018

[1] xxx

[2] xxx

[3] xxx

[4] CES_Basic_DID/PID_Specification

[5] CES_Basic_Diagnostics_Specification

[6] xxx

5 Lists

5.1 List of Tables

Table 1: Change History 4
Table 2: Rules for Versioning 4
Table 3: Abbreviations 5
Table 4: Example for the contents of VOD 8
Table 5: Example for the contents of CDS 8
Table 6: Structure of the VOD 10
Table 7: Structure of the CDS 10
Table 8: VOD message 19
Table 9: VOD block schedule 19

5.2 List of Figures

Figure 1: Classification of ECUs 7
Figure 2: Coding states of an ECU 12
Figure 3: Coding sequence for ECUs (storing Coding Data String) 14
Figure 4: Factory VOD and current VOD written 16
Figure 5: Current VOD written 17
Figure 6: Coding sequence for updating a vehicle's Vehicle Order Data 18
Figure 7: Strategy in ECU_A for mismatch detection 20
Figure 8: Strategy in ECU_B for mismatch detection 21

6 Further Applicable Documents

NR